



Mappa Riassuntiva dei Design Pattern (Esame Ingegneria del Software)



PATTERN CREAZIONALI

Risolvono i problemi legati alla creazione degli oggetti, nascondendo la logica di istanziamento.

1. Singleton

- **Definizione:** Garantisce che di una classe esista una sola istanza e fornisce un punto di accesso globale ad essa.
- **Correlati e Motivo:** * **Facade** e **Abstract Factory:** Ne basta quasi sempre una sola istanza per gestire l'intero sottosistema o le famiglie di oggetti, quindi sono spesso implementate come Singleton.
 - **State:** Se gli oggetti di stato concreto non conservano dati interni (variabili d'istanza), istanziarli più volte è uno spreco. Farli Singleton permette al Context di condividerli e risparmiare RAM.
- **Appunti da Lode:** Blocca la new col costruttore privato. Pericoli: Multithreading (risolto con synchronized) e Serializzazione (risolta con readResolve). La via moderna e più sicura in Java è usare le enum.
- **Differenze Chiave (vs Flyweight):** Il Singleton garantisce **una sola** istanza in assoluto per quella classe. Il Flyweight mantiene **molteplici** istanze condivise in base a uno stato intrinseco (es. una per ogni colore).

2. Prototype

- **Definizione:** Specifica i tipi di oggetti da creare usando un'istanza prototipale e crea nuovi oggetti copiando (clonando) questo prototipo.
- **Correlati e Motivo:** * **Command:** Molto usato per inserire un comando clonato nella "History List" (lo storico) in modo da mantenere intatto il comando originale collegato al bottone della UI.
 - **Composite:** Il Prototype viene spesso usato per clonare intere sezioni o rami complessi di una struttura Composite in modo rapido.
- **Appunti da Lode:** Evita gerarchie parallele e l'uso di new. Attenzione alla differenza tra *Shallow Copy* (copia riferimenti) e *Deep Copy* (copia oggetti annidati). Funziona perfettamente con oggetti immutabili (es. il Monomio).
- **Differenze Chiave (vs Factory Method):** Il Factory usa l'ereditarietà (una sottoclasse fa l'override del metodo di creazione). Il Prototype usa la composizione/clonazione: prende un oggetto *già esistente e configurato* in memoria e ne fa una fotocopia.

3. Factory Method

- **Definizione:** Definisce un'interfaccia per creare un oggetto, ma lascia alle sottoclassi

decidere quale classe istanziare.

- **Correlati e Motivo: * Template Method:** Il Factory Method è spesso richiamato come "step" all'interno di un Template Method quando l'algoritmo ha bisogno di generare un nuovo oggetto senza legarsi alla sua classe concreta.
 - **Iterator:** Le collezioni usano esattamente questo pattern (il metodo `createIterator()`) per restituire l'iteratore giusto in base alla loro implementazione interna.
- **Appunti da Lode:** È l'applicazione del Template Method alla creazione degli oggetti.
- **Differenze Chiave (vs Abstract Factory):** Il Factory Method è un **singolo metodo** che crea **un solo tipo** di oggetto tramite ereditarietà. L'Abstract Factory è un **oggetto** intero che crea una **famiglia** di oggetti correlati tramite composizione.

4. Abstract Factory

- **Definizione:** Fornisce un'interfaccia per creare *famiglie* di oggetti correlati o dipendenti senza specificare le loro classi concrete.
- **Correlati e Motivo: * Factory Method:** L'Abstract Factory stessa è spesso implementata internamente dichiarando un set di Factory Method (uno per ogni prodotto da creare).
 - **Singleton:** Un'applicazione di solito ha bisogno di una sola famiglia di prodotti attiva (es. solo l'interfaccia Windows), quindi l'Abstract Factory concreta è tipicamente un Singleton.
 - **Facade:** La Facade può usare l'Abstract Factory per istanziare gli oggetti del sottosistema, nascondendone del tutto la creazione al Client.
- **Appunti da Lode:** Il suo superpotere è garantire la **consistenza**: impedisce che vengano mischiati oggetti di famiglie diverse (es. pulsanti stile Mac in una finestra stile Windows).
- **Differenze Chiave (vs Builder):** L'Abstract Factory restituisce un'intera famiglia di oggetti *immediatamente*. Il Builder costruisce **un solo** oggetto complesso *passo dopo passo*.

5. Builder

- **Definizione:** Separa la costruzione di un oggetto complesso dalla sua rappresentazione, in modo che lo stesso processo di costruzione possa creare rappresentazioni diverse.
- **Correlati e Motivo: * Composite:** I Builder sono eccellenti per creare e assemblare le complesse strutture ad albero generate dal pattern Composite.
 - **Visitor:** Il Visitor può comportarsi come "Director" per il Builder: attraversa una struttura e, a ogni nodo che incontra, impartisce un ordine al Builder su cosa costruire (come nell'esempio del prof con i formati di testo).
- **Appunti da Lode:** Prevede due ruoli chiave: il **Director** (che sa *l'ordine* dei passaggi) e il **Builder** (che sa *come costruire* i pezzi). Fondamentale per evitare il costruttore "telescopico" con troppi parametri.
- **Differenze Chiave (vs Abstract Factory):** Il Builder si concentra sulla costruzione **passo per passo** di un oggetto complesso (ritorna l'oggetto solo alla fine col metodo `getResult()`). L'Abstract Factory restituisce i prodotti subito.



PATTERN STRUTTURALI

Risolvono i problemi di composizione di classi e oggetti per formare strutture più ampie.

6. Adapter

- **Definizione:** Converte l'interfaccia di una classe in un'altra interfaccia attesa dai client.
- **Correlati e Motivo:** * **Bridge:** Entrambi nascondono i dettagli, ma il Bridge si usa *a priori* (in fase di design) per far variare astrazione e implementazione, l'Adapter si usa *a posteriori* per adattare classi che altrimenti non dialogherebbero.
 - **Decorator:** L'Adapter cambia fisicamente la firma dei metodi per adattarli, mentre il Decorator mantiene le firme invariate aggiungendo solo funzionalità.
- **Appunti da Lode:** Modifica la FORMA (l'interfaccia) di un oggetto già esistente. **Non** supporta la composizione ricorsiva.
- **Differenze Chiave (vs Bridge e Facade):** L'Adapter fa funzionare insieme classi *esistenti* dopo che sono state progettate. La Facade crea una *nuova* interfaccia semplificata per un intero sottosistema, mentre l'Adapter adatta l'interfaccia di un *singolo* oggetto.

7. Decorator

- **Definizione:** Assegna dinamicamente nuove responsabilità a un oggetto.
- **Correlati e Motivo:** * **Composite:** Spesso usati insieme. Il Decorator può aggiungere comportamenti a un nodo del Composite. In effetti, il Decorator può essere visto come un Composite degenerato (che ha *un solo* figlio).
 - **Proxy:** Condividono la stessa identica interfaccia del Subject, ma per scopi diversi: il Decorator estende dinamicamente la funzionalità, il Proxy controlla le regole di accesso a tale funzionalità.
- **Appunti da Lode:** Non modifica l'interfaccia, cambia solo il COMPORTAMENTO interno. **Supporta** la composizione ricorsiva ("effetto matrioska"). L'alternativa flessibile alla derivazione di sottoclassi.
- **Differenze Chiave (vs Proxy e Adapter):** L'Adapter *cambia* l'interfaccia. Il Decorator *mantiene* la stessa interfaccia ma **aggiunge** funzionalità (es. aggiunge la scrollbar). Il Proxy *mantiene* l'interfaccia ma **controlla l'accesso** senza aggiungere funzionalità.

8. Composite

- **Definizione:** Compone gli oggetti in strutture ad albero (parte-tutto) per permettere ai client di trattare oggetti singoli e composizioni in modo *uniforme*.
- **Correlati e Motivo:** * **Visitor:** Si adatta perfettamente al Composite per eseguire operazioni trasversali su tutti i nodi dell'albero senza riempire l'interfaccia del Composite di metodi estranei.
 - **Iterator:** Fornisce il modo standardizzato per scorrere i nodi e le foglie dell'albero composito.
 - **Command:** Permette la creazione dei "MacroCommand", ovvero un comando-nodo che esegue a catena una lista di comandi-foglia.
- **Appunti da Lode:** Il grande Trade-off: Trasparenza (metodi gestione figli nella superclasse, ma viola Liskov) vs Sicurezza (metodi solo nel nodo, richiede cast).

- **Differenze Chiave (vs Decorator):** Entrambi usano la composizione ricorsiva, ma il Decorator ha **un solo** figlio e serve ad aggiungere comportamenti. Il Composite ha **molti** figli e serve a rappresentare aggregazioni (alberi).

9. Facade

- **Definizione:** Fornisce un'interfaccia unificata per un intero sottosistema, rendendolo più semplice da usare.
- **Correlati e Motivo:** * **Singleton:** In genere serve una sola Facciata per l'intero sottosistema, quindi viene istanziata come Singleton.
 - **Mediator:** Molto simili architetturealmente, ma la Facade astrae un intero sottosistema verso l'esterno in modo unidirezionale, mentre il Mediator astrae le comunicazioni interne tra oggetti in modo bidirezionale.
- **Appunti da Lode:** Comunicazione unidirezionale (Client -> Facade -> Sottosistema). Crea basso accoppiamento esterno, alta coesione interna. Le classi interne non sanno di essere dietro a una Facciata.
- **Differenze Chiave (vs Mediator):** La Facade è **strutturale**: semplifica l'uso di un sottosistema verso l'esterno. Il Mediator è **comportamentale**: centralizza la comunicazione tra oggetti *interni*.

10. Flyweight

- **Definizione:** Condivide oggetti per supportare in modo efficiente enormi quantità di oggetti a granularità fine.
- **Correlati e Motivo:** * **State** e **Strategy:** Entrambi questi pattern tendono a generare piccoli oggetti logici. Se tali oggetti vengono resi puri (ovvero conservano solo i metodi ma non tengono salvati i dati estrinseci), possono essere gestiti tramite Flyweight per riutilizzare le stesse identiche istanze in migliaia di contesti.
- **Appunti da Lode:** Divide lo stato in *Intrinseco* (immutabile, condiviso) ed *Estrinseco* (mutabile, calcolato e passato dal Client). Salva la RAM a discapito della CPU.
- **Differenze Chiave (vs Singleton):** Un Flyweight ha multipli oggetti condivisi in base allo stato intrinseco (es. una sola istanza del font Arial e una del font Times New Roman). Il Singleton ne ha rigorosamente una sola in assoluto.

11. Bridge

- **Definizione:** Disaccoppia un'astrazione dalla sua implementazione, in modo che le due gerarchie possano variare indipendentemente.
- **Correlati e Motivo:** * **Abstract Factory:** Può essere combinata con il Bridge per delegare la creazione dell'oggetto "Implementazione" corretto in base all'ambiente di esecuzione (es. far creare alla Factory l'Implementazione Windows della FinestraBridge).
- **Appunti da Lode:** Evita l'esplosione combinatoria delle sottoclassi. Spezza l'oggetto in due gerarchie (Astrazione e Implementazione) collegate per Composizione (il "Ponte").
- **Differenze Chiave (vs Adapter):** Il Bridge si applica **durante** la fase di design per mantenere due concetti separati. L'Adapter si usa **dopo** che le classi sono state

implementate per farle parlare.

12. Proxy

- **Definizione:** Fornisce un surrogato o un segnaposto per un altro oggetto, per controllarne l'accesso.
- **Correlati e Motivo:** * **Decorator:** Hanno scopi diversi ma implementazione identica (un oggetto "scatola" che implementa la stessa interfaccia del suo "contenuto" inoltrandogli le chiamate).
- **Appunti da Lode:** Tipi principali: Virtual Proxy (creazione *lazy* per oggetti pesanti), Remote Proxy, Protection Proxy (sicurezza).
- **Differenze Chiave (vs Decorator):** Il Proxy si finge l'oggetto originale per gestirne il ciclo di vita (crearlo solo quando serve) o l'accesso (protezione). Il Decorator si attacca all'oggetto originale *già esistente* per fornirgli nuove abilità.

PATTERN COMPORTAMENTALI

Risolvono i problemi legati all'assegnazione delle responsabilità e agli algoritmi tra gli oggetti.

13. Observer (Publish-Subscribe)

- **Definizione:** Dipendenza 1-a-N. Se un oggetto (Subject) cambia stato, notifica automaticamente tutti i suoi dipendenti (Observer).
- **Correlati e Motivo:** * **Mediator:** Il Mediator usa spesso l'Observer come meccanismo "sottobanco": i colleghi agiscono da Subject (pubblicando un evento) e il Mediator fa da Observer per intercettarlo e reagire, garantendo che i colleghi non debbano mai conoscersi.
 - **Memento:** Quando l'Originator usa il Memento per fare l'Undo e ripristinare il vecchio stato, può lanciare una notifica Observer per segnalare alla UI che i dati sono cambiati (come nell'esempio dell'editor grafico del prof).
- **Appunti da Lode:** Push model (invia dati) vs Pull model (invia solo avviso).
- **Differenze Chiave (vs Mediator):** Nell'Observer gli oggetti comunicano a "rete" in modo decentralizzato. Nel Mediator la comunicazione è centralizzata (tutti parlano solo col Direttore).

14. Mediator

- **Definizione:** Incapsula e centralizza le modalità di interazione tra un gruppo di oggetti (Colleghi).
- **Correlati e Motivo:** * **Observer:** Utilizzato dal Mediator stesso per ricevere notifiche dai Colleagues passivamente, riducendo l'accoppiamento tra il Mediator e i suoi "impiegati".
 - **Facade:** Hanno scopi opposti (interno vs esterno).
- **Appunti da Lode:** Trasforma comunicazioni a "spaghetti" (N-a-N) in comunicazione a "stella" (1-a-N). Svantaggio: il Mediator può diventare un "God Object" difficile da mantenere.
- **Differenze Chiave (vs Facade e Observer):** A differenza della Facade, il Mediator

coordina la logica di business *tra* i colleghi ed è conosciuto da essi. A differenza dell'Observer, qui la logica di coordinamento è esplicitata fisicamente in una classe centrale.

15. State

- **Definizione:** Permette a un oggetto (Context) di cambiare il suo comportamento al variare del suo stato interno (come se cambiasse classe).
- **Correlati e Motivo:** * **Singleton** e **Flyweight:** Dato che in un sistema ci sono continui passaggi di stato (es. Play -> Pausa -> Play), le classi di Stato Concreto vengono spesso rese Singleton o Flyweight per non dover allocare nuova memoria (fare new) a ogni singolo click dell'utente.
- **Appunti da Lode:** Elimina giganteschi blocchi if/else per controllare i flag di stato interni.
- **Differenze Chiave (vs Strategy):** Il diagramma UML è identico, l'intento è opposto. Nello **State**, gli stati sono consapevoli l'uno dell'altro e causano le *transizioni* in automatico. Nello **Strategy**, il Client esterno sceglie e inietta la strategia dall'alto.

16. Strategy

- **Definizione:** Incapsula una famiglia di algoritmi rendendoli intercambiabili, separandoli dal client che li utilizza.
- **Correlati e Motivo:** * **Flyweight:** Come per lo State, se crei oggetti Strategy che contengono solo il calcolo dell'algoritmo senza mantenere variabili, essi possono essere condivisi in memoria come Flyweight.
 - **Template Method:** Risolvono lo stesso identico problema architetturale (isolare gli algoritmi), ma uno usa la Composizione, l'altro l'Ereditarietà.
- **Appunti da Lode:** Mantengo fisso il problema, cambio l'algoritmo. Svantaggio: il Client deve conoscere le differenze tra le varie strategie per poterle scegliere.
- **Differenze Chiave (vs Template Method):** Lo Strategy altera l'intero algoritmo delegandolo a un oggetto esterno (Composizione). Il Template Method altera **alcuni passi** dell'algoritmo usando le sottoclassi (Ereditarietà).

17. Template Method

- **Definizione:** Definisce lo scheletro di un algoritmo in un metodo, delegando i passi specifici alle sottoclassi (ereditarietà).
- **Correlati e Motivo:** * **Factory Method:** Se uno dei passaggi che il Template Method delega alla sottoclasse riguarda il "creare un oggetto", ecco che quel particolare passaggio delegato si trasforma esattamente in un Factory Method.
- **Appunti da Lode:** Mantengo fisso l'algoritmo, cambio il problema. Basato sul Principio di Hollywood: "Non chiamarci tu, ti chiamiamo noi".
- **Differenze Chiave (vs Strategy):** Usa l'ereditarietà di classe (white-box) anziché la composizione di oggetti (black-box). Lo scheletro dell'algoritmo non cambia mai, cambiano solo i passaggi implementati nelle ConcreteClass.

18. Command

- **Definizione:** Incapsula una richiesta/azione in un oggetto indipendente.
- **Correlati e Motivo: * Composite:** Incastrandoli, si creano i famosi "MacroCommand": un comando primario che contiene ed esegue una lista di comandi figli.
 - **Memento:** Si usano insieme per l'opzione "Annulla" (Undo). Prima che il Command esegua il suo `dolt()`, si fa dare una fotografia dello stato tramite il Memento. Se richiamato con `undolt()`, il Command restituisce il Memento al Receiver per ripristinarlo.
- **Appunti da Lode:** Disaccoppia l'Invoker (es. un tasto UI) dal Receiver (il documento). Rende possibili logiche di Undo/Redo e le code di richieste.
- **Differenze Chiave (vs Strategy):** Un *Command* rappresenta una "richiesta specifica" congelata (es. "Salva File pippo.txt"). Una *Strategy* rappresenta "il modo in cui si fa qualcosa" (es. "Algoritmo di salvataggio ZIP o RAR").

19. Memento

- **Definizione:** Cattura lo stato interno di un oggetto (per poterlo ripristinare dopo) senza violare l'incapsulamento.
- **Correlati e Motivo: * Command:** Il Memento è l'oggetto "carburante" che permette all'oggetto Command di riavvolgere il tempo in totale sicurezza, salvaguardando i dati privati dell'Originator.
- **Appunti da Lode:** Tre ruoli: Originator (crea), Memento (scatola nera intoccabile), Caretaker (lo conserva ma non può leggerlo). Attenzione al costo della RAM per mantenere lo storico nel Caretaker.
- **Differenze Chiave (vs Command):** Il Command memorizza l'azione da fare o disfare. Il Memento memorizza una fotografia fisica dello stato (i dati privati dell'oggetto) prima e dopo l'azione.

20. Chain of Responsibility

- **Definizione:** Disaccoppia mittente e ricevente passando la richiesta lungo una catena di oggetti finché uno non la gestisce.
- **Correlati e Motivo: * Composite:** I legami "padre-figlio" di una struttura Composite sono ideali per fare da tracciato alla catena. Se una "foglia" (es. un Widget grafico) non sa gestire il click del mouse, lo passa al suo contenitore "nodo padre", e così via fino alla finestra principale.
- **Appunti da Lode:** La catena è dinamica (modificabile a runtime alterando i puntatori next). Il problema architetturale è che *non c'è garanzia* che la richiesta venga processata: se arriva a fine catena, muore silente.
- **Differenze Chiave (vs Decorator e Observer):** Un Decorator aggiunge un comportamento e *continua sempre* a chiamare l'oggetto avvolto. La Chain passa la palla solo se non sa gestire la richiesta. L'Observer notifica *tutti*; la Chain ne fa gestire *uno solo*.

21. Visitor

- **Definizione:** Rappresenta un'operazione da eseguire sugli elementi di una struttura, permettendo di aggiungere operazioni senza modificare le classi.
- **Correlati e Motivo: * Composite:** Architetture gemelle in fase di esecuzione. Il Visitor è il pattern di riferimento per processare e analizzare strutture ad albero complesse generate dal Composite.
 - **Builder:** Mentre attraversa il Composite, il Visitor può assumere le redini come "Director", comandando a un Builder come ricostruire o tradurre l'albero in un altro formato (es. esportazione da Documento a HTML).
- **Appunti da Lode:** Usa l'*Inoltro doppio* (Double Dispatch). Vantaggio: facile inventare nuove operazioni creando un nuovo Visitor. Svantaggio: difficilissimo inventare nuovi *tipi* di Elementi alla struttura.
- **Differenze Chiave (vs Iterator):** L'Iterator scorre collezioni di elementi omogenei e si disinteressa di cosa tu ci faccia dopo. Il Visitor scorre strutture di elementi eterogenei per applicare una complessa operazione predefinita su ciascuno di essi.

22. Iterator

- **Definizione:** Fornisce accesso sequenziale agli elementi di un contenitore aggregato senza esporre la sua struttura interna.
- **Correlati e Motivo: * Factory Method:** Il metodo `createIterator()` richiesto dalle collezioni aggregate è il classico Factory Method.
 - **Composite:** Spesso viene implementato un iteratore apposito per permettere al client di scorrere linearmente (in modalità "piatta") una struttura complessa ad albero Composite senza doversi occupare della ricorsione.
- **Appunti da Lode:** Sposta lo stato dell'attraversamento (il cursore, l'indice *i*) fuori dalla collezione, permettendo più attraversamenti simultanei sulla stessa lista senza che vadano in conflitto.
- **Differenze Chiave (vs Visitor):** L'Iterator ti consegna i dati passivamente (pull). Il Visitor impone attivamente il suo comportamento ai dati che visita (push).

23. Interpreter

- **Definizione:** Definisce una rappresentazione per una grammatica (albero sintattico) e l'interprete per valutarla.
- **Correlati e Motivo: * Composite:** L'Abstract Syntax Tree (AST) che viene generato è fisicamente implementato col pattern Composite. I simboli non-terminali (es. un blocco logico "AND") sono i Nodi, i simboli terminali (es. la stringa da cercare) sono le Foglie.
 - **Visitor:** Al posto di mettere il codice di esecuzione del comando dentro i singoli nodi dell'albero AST, si può usare il Visitor per attraversare l'albero e valutare le espressioni grammaticali esternamente.
- **Appunti da Lode: NON fa analisi lessicale o sintattica.** Riceve l'albero AST già costruito in memoria (da un Lexer/Parser esterno) e lo interpreta viaggiando di nodo in nodo.
- **Differenze Chiave (vs Composite):** La differenza non è nella struttura in memoria (identica), ma nell'intento: l'Interpreter modella regole grammaticali formali al fine di

risolvere/tradurre un'espressione finale (spesso calcolando un valore di ritorno di ritorno).



PATTERN ARCHITETTURALI / BASE

24. Data Transfer Object (DTO)

- **Definizione:** Un oggetto usato per trasferire aggregati di dati tra oggetti indipendenti o reti.
- **Correlati e Motivo:** * **Mapper / Assembler:** Lavorano insieme. Il DTO è la "valigia" passiva, mentre il Mapper/Assembler è lo "scalo merci" attivo che si occupa di prendere il Domain Object originario, spaccettarlo e travasarne solo i dati utili e sicuri all'interno del DTO.
- **Appunti da Lode:** È un oggetto anemico (nessuna logica di business, solo attributi e get/set). Evita la congestione della rete limitando le chiamate API ripetute.
- **Differenze Chiave (vs Domain Object):** Il Domain Object è un oggetto "ricco" (logica di business, metodi complessi, query verso il database). Il DTO è una scatola piatta progettata esclusivamente per attraversare canali di rete senza scatenare crash o falle di sicurezza.



MECCANISMI FONDAMENTALI OOP

Non sono Design Pattern, ma sono gli strumenti primari su cui tutti i pattern si basano.

A. Ereditarietà (Inheritance)

- **Definizione:** Classe base cede caratteristiche a classe derivata. Relazione "**IS-A**" (è un).
- **Vantaggi/Svantaggi:** Permette *White-box reuse* (puoi sovrascrivere l'implementazione del padre). È rigida (definita per sempre a *compile-time*). Introduce un forte accoppiamento genitore-figlio e rompe parzialmente l'incapsulamento. Pattern simbolo: *Template Method* e *Factory Method*.

B. Delegazione / Composizione (Delegation / Composition)

- **Definizione:** Un oggetto delega un compito o un'informazione a un altro oggetto che conserva al suo interno come variabile. Relazione "**HAS-A**" (ha un).
- **Vantaggi/Svantaggi:** Permette *Black-box reuse* (usa i metodi esposti dall'oggetto contenuto senza conoscerne l'implementazione interna). Mantiene basso l'accoppiamento, preserva l'incapsulamento ed è dinamica: puoi cambiare l'oggetto contenuto "al volo" durante la *run-time*. È il pilastro assoluto dell'Ingegneria del Software (*Strategy, State, Decorator*).